LLM Router Research

Nov 6, 2026 · Harry

Build three routers, in this order: Arch-Router, Avengers-Pro, CARROT. Picking the best model per task type scores 0.814 against 0.793 for the best single model, and no query-level router we tested beat that pick. So the first router classifies the request; the next two go after cost and latency. Scores here come from `Results/results.db`, whose rows now include Gemini on NegotiationToM.

What changed since the Nov 4 version: the best single model is now Deepseek, not Gemma, because scores use the reported weighting rather than counting every row alike. The router order stands. The expected gain rises from about 1.5 to about 2 points. A wrong category is no longer cheap everywhere: on AwareBench it costs 12 points.

Why the numbers differ between documents

Which model leads depends on how scores are averaged, so every table below states its basis. This document uses the second row: it is the reported scores, on the five benchmarks all models can run.

Basis (mean over benchmarks)	Gemma	Deepseek	Gemini	Luna	Qwen
Every row alike, 5 benchmarks (Nov 4 version)	**0.811**	0.803	0.794	0.793	0.756
Reported weighting, 5 benchmarks (this document)	0.781	**0.793**	0.771	0.772	0.721
Workbook scores with All_EM, 5 benchmarks	0.698	**0.704**	0.686	0.682	0.636
Workbook scores with All_EM and DocVQA ANLS, 6 benchmarks	**0.736**	text-only	0.728	0.712	0.690

Three things drive the differences:

- **AwareBench.** Counting every row gives Gemma 0.931, inflated by rows outside the reported score: 900 of the 4,015 are a multiple-choice perspective task scored 1.000 by all five. The reported score leaves those out and gives second-order perspective a fifth of the weight, where Gemma scores 0.129 (it declined 124 of 170) and Deepseek 0.806. Gemma's reported score is 0.790.
- **NegotiationToM.** It has no overall score. All_EM, the last row of its sheet, is a separate metric: a dialogue counts only if all its desire, belief and intention rows are right, usually six, so it sits at 0.04–0.09. It is not a total, so **the two All_EM rows above are invalid as summaries and are shown only to reconcile with averages that used it.** This document uses the mean of desire, belief and exact-match intention instead, because a router can score those per query. Desire and belief equal the database; **exact-match intention is the one term here that is not a database number.**
- **DocVQA.** Deepseek cannot read images, so it has no six-benchmark mean.

The weighted rows reproduce each reported overall score, and NegotiationToM's desire and belief; `router_reported.py` stops if they do not.

Is our data enough?

For routing on accuracy, yes: six benchmarks have a per-query score that joins across models. For routing on cost and on real traffic, not yet.

Benchmark	Queries scored for all models	Tokens + latency in the database
NegotiationToM	13,826	Luna, Gemini
DocVQA	5,349 (four models; Deepseek is text-only)	none
bbh	4,833	none
AwareBench	4,015 (2,753 carry weight in the reported score)	all five
MMLU	3,943	none
EmoBench	400	Luna (Gemini logged tokens, not latency)
MultiChallenge, Wonderbread	generated (Wonderbread without Deepseek); judging started Nov 4, no scores yet	logged at generation; not yet in the database

Gaps to close before training:

- **Cost.** Prices are recorded for all five models, but usage exists only as shown above, so the only benchmark with all five costed is AwareBench. Qwen's new runs moved to DeepInfra, so its price needs reading again for that route.
- **Real requests.** Every row is a benchmark item. Collect 500–2,000 requests shaped like real use, answer them with all five models and grade them with the Haiku 5.5 judge (estimate: $10–40 to generate, ~$5 to judge).
- **Consistency.** MMLU mixes two scorers and two prompt versions, which matters because its top two models are 0.0002 apart. The legacy model behind `deepseek-reasoner` was retired on 2026-07-24, so confirm which model produced the older Deepseek rows; AwareBench's Deepseek column was called as `deepseek-flash`.

What can routing gain on our data?

About 2 points of macro accuracy, and picking by task type already captures it.

Benchmark	Best model	Runner-up	Query-level oracle	Models disagree
NegotiationToM	Luna 0.527	Gemini 0.514	0.699	41% of items
EmoBench	Gemma 0.745	Gemini 0.698	0.873	39%
AwareBench	Deepseek 0.916	Gemini 0.794	0.960	35%
MMLU	Luna 0.918	Deepseek 0.917	0.964	16%
DocVQA (accuracy)	Qwen 0.971	Gemini 0.969	0.990	11%

Benchmark	Best model	Runner-up	Query-level oracle	Models disagree
bbh	Gemma 0.968	Deepseek 0.962	0.983	11%

Reported weighting; the oracle and disagreement are weighted the same way. NegotiationToM has no single winner in the database: Luna leads desire and belief, Gemma leads intention F1, and Gemini leads All_EM.

Pooled over the five benchmarks all models ran, with every trained router scored on held-out folds:

Router (macro accuracy)	Score
Always Qwen	0.721
Always Gemini	0.771
Always Luna	0.772
Always Gemma	0.781
Always Deepseek (best single model)	0.793
Query-level kNN, k=10 / 50 / 100	0.802 / 0.810 / 0.811
Query-level logistic regression	0.810
Table: best model per benchmark	0.814
Table: best model per sub-task	0.816
Oracle (ceiling)	0.896

Computed from per-query result files, Nov 6, 2026 · 5-fold grouped cross-validation · TF-IDF features · 27,017 queries · confidence intervals from a paired bootstrap of 2,000 draws over dialogues and questions.

Difference, in points	Gap	95% interval	Reading
Benchmark table vs always Deepseek	+2.2	1.4 to 3.0	routing gains
Logistic regression vs always Deepseek	+1.8	0.9 to 2.6	query-level gains too
Benchmark table vs kNN, k=100	+0.3	−0.2 to 0.9	no detectable difference
Benchmark table vs logistic regression	+0.4	−0.3 to 1.1	no detectable difference
Benchmark table vs sub-task table	−0.2	−0.5 to 0.2	no detectable difference
Always Deepseek vs always Gemma	+1.2	0.2 to 2.1	small, and from one benchmark

What this does and does not show. TF-IDF query-level routers did not beat a lookup table, and the table needs no training. Embedding and larger query-level routers are untested on our data, so this does not rule them out.

Deepseek's lead rests on AwareBench. Over the other four benchmarks Gemma leads, 0.779 to 0.762. So treat the two as a close pair rather than a settled ranking, and keep both as baselines.

Part of the gap to the oracle is noise, in an amount we have not measured. The oracle credits whichever model happened to be right. Rerunning Luna unchanged changes its answer on 18% of the NegotiationToM subsample and 11% of EmoBench's (7–21% by task), while Gemini's NegotiationToM reruns are byte-identical. Rewording alone changes about a quarter of NegotiationToM answers for both (Luna 23%, Gemini 25%).

What it costs

Only AwareBench has all five costed. The most accurate model is also the most expensive per correct answer, by about five times over the two cheapest.

Model	AwareBench score	$ per 1k correct	p50 latency (s)	Reasoning tokens per row
Deepseek	0.916	0.116	1.06	150
Gemini	0.794	0.067	0.64	0
Luna	0.791	0.050	0.76	15
Gemma	0.790	0.024	1.41	0
Qwen	0.691	0.023	0.30	not reported

On NegotiationToM, the only other benchmark with two models costed, Luna costs $0.383 per 1k correct against Gemini's $0.252, for 1.3 points more. Rates are as read on Oct 12, 2026; Deepseek's price varies by hour.

Three routers for this project

#	Router	What it does	Why it fits us	Training
1	Arch-Router (Katanemo, 1.5B)	Matches a request to policies we write, one per task type, and returns the model tied to that policy	The best model per benchmark scored 0.814 vs 0.793 for always-Deepseek, and no query-level router we tested did better. Arch-Router makes that pick with no training, and its decisions are readable	None: pretrained
2	Avengers-Pro	Embeds requests, groups them into ~60 clusters, scores each model per cluster as α·accuracy + (1−α)·(1−cost)	Adds cost to the choice: where models tie it picks the cheaper one. Ties are common on three of six benchmarks (models disagree on only 11–16% of items), and cost per correct answer differs about fivefold	Clustering only: minutes on a laptop
3	CARROT (kNN variant)	Predicts each model's chance of being right and its cost for this request, then picks the best trade-off	Cost varies per request, not only per model: Deepseek averages 150 reasoning tokens per AwareBench row but about 1,465 on second-order perspective items	kNN: minutes; RoBERTa variant ~1 GPU-hour

Considered and not chosen as a first build:

Router	Why not first	Training
RouteLLM	Routes between two models only (strong vs weak); five need a wrapper	Minutes
IRT-Router	Gives interpretable ability scores per model, useful for the write-up rather than for routing gains	Minutes to 1 h
Not Diamond	Closed, at most 10,000 rows per training job; keep it as the commercial baseline	Hosted, minutes to 1 h
GraphRouter, RouterDC	More training with no clear payoff: tuned kNN matched or beat learned routers, including a GraphRouter-inspired one, in this comparison (RouterDC was not in its experiments), and several recent routers did not reliably beat the best single model in LLMRouterBench	1–3 GPU-hours
FrugalGPT	A cascade of sequential calls adds latency	~1 GPU-hour
PickLLM	Ignores the request and settles on one model per session	Online
MoA, LLM-Blender, DeePEn, FuseLLM	Ensembles: 2–6 calls per request, or need token probabilities and open weights	—

How to implement

1. **Build one training table first (all three routers use it).** One row per query × model: query text, benchmark, score, tokens, latency, cost. 27,017 queries are ready. Close the gaps listed above first, and add MultiChallenge and Wonderbread once their judge scores are in the database. Store the reported weight with each row so a router is trained and scored on the same basis.
2. **Arch-Router.** Write one policy per task type (a name and a one-line description), not one per taxonomy category: the interpersonal category alone has two different best models, and the task category has three. Map each to its best model from `Results/results.db`, and serve the router from Hugging Face on a Quest GPU. A request with no confident match goes to the fallback model below.
3. **Avengers-Pro.** Use the authors' code. Embed the queries, cluster with k-means ($k \approx 60$), and store each model's accuracy and cost per cluster. Sweep α to choose the cost–accuracy point.
4. **CARROT.** On the same embeddings, a kNN ($k \approx 50$–100) predicts each model's P(correct) and cost. Route to the model with the best $P - \lambda \cdot \text{cost}$, and sweep λ.
5. **Test before adopting.** Score all three on the same held-out folds against always-Deepseek, always-Gemma and the plain table. Keep a router only if it is more accurate, or as accurate for less cost or latency, beyond the paired-bootstrap interval.

How to tell it works

- **Baselines:** always-Deepseek and always-Gemma (the close top pair), always the cheapest model, random, the plain table, and the oracle as the ceiling.
- **Splits:** grouped 5-fold cross-validation (a dialogue or question never on both sides), leave-one-benchmark-out for unseen task types, and the realistic-request set as the final test. Unseen

task types are the hard case: with a whole benchmark held out, kNN (k=50) falls 0.7–11.6 points short of that benchmark's best model.

- **Metrics:** macro accuracy on the reported weighting, with benchmarks weighted equally, cost per 1,000 requests, and median and p90 latency. Sweep α or λ to draw the cost–accuracy curve.
- **Noise floor:** the paired bootstrap above, per comparison. For scale, Luna's six EmoBench runs span 0.658–0.688.

Can a small router recognise the request?

Recognising the *kind* of request is easy on our data. A TF-IDF + logistic-regression classifier, scored on held-out folds, names the benchmark with 0.998 accuracy and the sub-task (48 classes) with 0.977. Most of its errors are MMLU subjects confused with each other. Treat 0.998 as an upper bound: benchmark items have distinctive formats, and real requests that mix categories, or fall between mission analysis and goal specification, will be harder. Measure it on the realistic-request set before relying on it.

A wrong category is cheap on most benchmarks and expensive on one. Sending a request to the second-best model instead of the best costs 0.0 points on MMLU, 0.7 on bbh, 1.3 on NegotiationToM, 4.8 on EmoBench and 12.2 on AwareBench. The worst model, Qwen on all five, costs 3.5–22.5 points.

Fallback below the confidence threshold: Gemma, as a judgement rather than a measurement. Deepseek scores 1.2 points higher, but that lead comes from one benchmark, it costs about five times as much per correct answer on AwareBench, and it cannot read images. Revisit this once the realistic-request set shows what low-confidence requests look like.

Reproduce

All scripts read stored results and call no model; on a laptop each takes a few minutes at most:

```
/opt/miniconda3/bin/python3 -I Router_research/router_reported.py   # scores, intervals → reported.json
/opt/miniconda3/bin/python3 -I Router_research/router_headroom.py   # every row alike → headroom.json
/opt/miniconda3/bin/python3 -I Router_research/intent_check.py      # task-type classifier
/opt/miniconda3/bin/python3 Router_research/build_pdf.py            # → Results/ROUTER_RESEARCH.pdf
```

Cost, latency, token and rerun figures are read from `Results/results.db`, except raw rewording flips and second-order reasoning tokens, which come from each model's `criteria_summary.json` and the workbook's Provenance notes.

Sources

Arch-Router · Avengers-Pro · CARROT · LLMRouterBench · kNN routers · Not Diamond. Not opened: RouteLLM, IRT-Router, FrugalGPT, PickLLM and the ensemble methods, whose rows above rest on what is generally known about them. Training times and the realistic-request costs are estimates. All accuracy figures for our models were computed offline from our per-query result files and checked against `Results/results.db`.